Advanced Day 12 — Session 12 (Hours 45–48)

Python Programming (Advanced) • Testing, Packaging, and Final Delivery

Session 12 Overview

Topics Covered Today

- Hour 45 — Unit tests for the service layer with pytest
- Hour 46 — Coverage, edge cases, and a lightweight integration test
- Hour 47 — Packaging and delivery: requirements, entry point, fresh-run test
- Hour 48 — Final capstone demo, certification-style review, and retrospective

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour1_Advanced.md` → `# Day 12, Hour 1: Testing with pytest: unit tests for the service layer`
- `Advanced/lessons/lecture/Day12_Hour2_Advanced.md` → `# Day 12, Hour 2: Coverage, edge cases, and a lightweight integration test`

Session 12 Outcomes

By the end of today, learners will be able to:

- Write focused pytest unit tests for service-layer logic using fixtures and plain `assert`
- Measure test coverage and identify high-value missing branches
- Add edge-case and integration-style tests that prove real layer boundaries
- Produce a runnable deliverable with a clear README and dependency files
- Deliver a concise, evidence-based final capstone demo
- Name one concrete next-step practice goal after the course

Scope Guardrails for Today

In Scope

- Service-layer and repository unit tests with pytest fixtures and `tmp_path`
- Coverage as a risk guide — not a vanity metric
- `README.md`, `requirements.txt`, `requirements-dev.txt`, and entry points
- Timeboxed final capstone demo with architecture explanation
- Certification-style code-reading review

Not Yet

- Testing Flask routes exhaustively (service-layer tests are the priority here)
- Chasing 100% coverage as a goal
- Production wheels, PyInstaller executables, or full CI/CD pipelines

Hour 45 — Unit Tests for the Service Layer

Session 12, Hour 45

Hour 45 Learning Outcomes

By the end of this hour, learners will be able to:

- Write unit tests for core service logic using pytest
- Use `pytest.fixture` and `tmp_path` for shared setup and isolated databases
- Assert both happy paths and expected exceptions with `pytest.raises`
- Run `pytest -q` and read failures as feedback, not judgment

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour1_Advanced.md` → `## Learning Outcomes`

Why Testing Fits Here

The capstone now has enough stable structure to test meaningfully.

- Service logic and exception patterns are established enough to protect
- Tests protect behavior while we make final changes and packaging decisions
- Running `pytest -q` before each session gives a clean, trustworthy baseline
- A failing test is information, not an accusation

> "Make one small change, run it, observe the result, then continue."
>
> — `Advanced/lessons/lecture/Day12_Hour1_Advanced.md` → `## Opening Script`

unittest → pytest Bridge

If you have seen `unittest` before, here is the direct mapping:

unittest habit	pytest equivalent
`class TestThing(unittest.TestCase)`	Plain `def test_thing():` functions
`self.assertEqual(actual, expected)`	`assert actual == expected`
`setUp()` / `tearDown()`	Fixtures that return prepared objects
`python -m unittest`	`pytest -q`

pytest does not ask you to forget Arrange → Act → Assert.
It just lets the test read like ordinary Python.

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour1_Advanced.md` → `### unittest to pytest`

Fixture + tmp_path Pattern

```python
import pytest
from tracker.repository import SQLiteTrackerRepository
from tracker.service import TrackerService, ValidationError

@pytest.fixture
def service(tmp_path):
    repo = SQLiteTrackerRepository(str(tmp_path / "test_tracker.db"))
    repo.init_db()
    return TrackerService(repo=repo)
```

- `tmp_path` is a built-in pytest fixture — a fresh isolated directory per test run
- No shared databases — every test starts with a clean slate
- `init_db()` creates the schema before any assertion runs

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour1_Advanced.md` → `### Demo code or command`

Demo: Happy-Path Test

```python
def test_create_and_get_record(service):
    created = service.create_record(
        {"title": "Demo", "category": "Lab", "status": "open"}
    )
    loaded = service.get_record(created.id)
    assert loaded.title == "Demo"
    assert loaded.category == "Lab"
    assert loaded.status == "open"
```

Arrange — Act — Assert

- **Arrange** — fixture built a clean service with an isolated database
- **Act** — call `create_record`, then `get_record`
- **Assert** — each field matches exactly what was stored

Source Alignment

Demo: Sad-Path Test

```python
def test_blank_title_raises_validation_error(service):
    with pytest.raises(ValidationError):
        service.create_record(
            {"title": "   ", "category": "Lab", "status": "open"}
        )
```

- `pytest.raises` is the correct pattern for asserting expected exceptions
- Do **not** catch the exception in the test body — that hides the failure
- Negative tests matter as much as happy paths

> Every feature is incomplete until the sad path is also specified and tested.

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour1_Advanced.md` → `### Demo code or command sketch`

Lab: Write the First Real Test Suite (Hour 45)

Time — 25–35 minutes

Tasks

1. Create `tests/test_service.py`
2. Write at least five tests — include at least two negative tests
3. Use `tmp_path` so every test runs with an isolated database
4. Run `pytest -q` — confirm all tests pass

Completion Criteria

- Test names communicate the behavior being checked
- Exceptions are asserted with `pytest.raises`, not caught and swallowed
- Tests run without the GUI, Flask server, or shared production state

Common Pitfalls (Hour 45)

⚠️ Testing UI callbacks or Flask routes instead of core service logic

⚠️ One test that checks too many behaviors at once — split into focused tests

⚠️ Shared mutable state — one test passes, the next fails unpredictably

⚠️ Catching exceptions in the test body instead of asserting them with `pytest.raises`

⚠️ Happy-path-only thinking — missing records and invalid input are equally important

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour1_Advanced.md` → `### Common pitfalls to watch for`

Quick Check (Hour 45)

Question: What should a well-written unit test avoid in order to stay fast and safe to refactor?

- A — A real database shared between all tests
- B — Plain `assert` statements
- C — A `pytest.fixture` for setup
- D — Checking a sad path with `pytest.raises`

> **Answer: A** — shared real databases make tests order-dependent and slow. Options B, C, and D are all recommended pytest patterns.

Hour 46 — Coverage, Edge Cases, and Integration

Session 12, Hour 46

Hour 46 Learning Outcomes

By the end of this hour, learners will be able to:

- Run a test coverage report and interpret it responsibly
- Choose high-value edge cases based on risk, not uncovered line counts
- Write one integration-style test that exercises service and repository together
- Explain why high coverage can still leave real bugs uncovered

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour2_Advanced.md` → `## Learning Outcomes`

Coverage: Flashlight, Not Scorecard

Coverage tells us **what code executed** during tests.

- A line that ran is not a line that is necessarily correct
- A test that touches a line without a meaningful assertion adds false confidence
- Chasing 100% produces low-value tests that slow down future refactoring

The right question to ask

Which uncovered branch is most likely to contain a real bug based on risk and past failures?

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour2_Advanced.md` → `### Talk points`

Running Coverage

```
pip install pytest-cov
pytest --cov=tracker --cov-report=term-missing -q
```

Reading the report

- The `Miss` column shows line numbers that never executed during tests
- Focus on logic branches — conditions, error paths, and edge inputs
- Ignore boilerplate lines and generated glue code

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour2_Advanced.md` → `### Demo steps`

High-Value Edge-Case Candidates

Scenario	Why it matters
Record not found — `NotFoundError`	Most real queries involve unknown or deleted IDs
Blank or whitespace-only title	Validation must catch invisible bad input
Invalid status value	Enum-style rules fail silently without a check
Persistence across two service instances	Proves SQLite actually committed the row
Duplicate or boundary IDs	Integer overflow and off-by-one are common bugs

Source Alignment

Demo: Not-Found Edge Case

```python
from tracker.service import NotFoundError

def test_missing_record_raises(service):
    with pytest.raises(NotFoundError):
        service.get_record(999_999)
```

The underscore separator in `999_999` is valid Python 3 — it improves readability of large integers.

Demo: Integration Test — Persistence Across Instances

```python
def test_record_persists_across_service_instances(tmp_path):
    db_path = tmp_path / "tracker.db"
    first_svc = build_test_service(db_path)
    created = first_svc.create_record(
        {"title": "Persist me", "category": "Lab", "status": "open"}
    )

    second_svc = build_test_service(db_path)
    loaded = second_svc.get_record(created.id)

    assert loaded.title == "Persist me"
    assert loaded.status == "open"
```

This test exercises the service **and** repository boundary with a real SQLite file — not a mock.

Source Alignment

Lab: Harden the Suite (Hour 46)

Time — 25–35 minutes

Tasks

1. Run `pytest --cov=tracker --cov-report=term-missing -q`
2. Identify at least two uncovered branches that carry real risk
3. Write two targeted tests for those branches
4. Write one integration-style test using a real `tmp_path` SQLite database

Completion Criteria

- Coverage improves in risk-informed, meaningful areas
- At least one integration-style test passes
- Edge cases include not-found and at least one validation boundary

Common Pitfalls (Hour 46)

⚠️ Chasing 100% with tests that `assert True` or check only that code executed

⚠️ Turning an integration test into a full end-to-end workflow — keep it narrow

⚠️ Ignoring nondeterministic values such as timestamps and auto-generated IDs

⚠️ Using production database files in tests instead of `tmp_path`

⚠️ Testing Flask route internals when a pure-function unit test would be simpler

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour2_Advanced.md` → `### Common pitfalls to watch for`

Quick Check (Hour 46)

Question: A coverage report shows 92% line coverage. How can the remaining gaps still contain a real bug?

> **Discussion points:**
>
> - Covered lines may have no meaningful assertion — code ran but nothing was checked
> - Branches inside covered lines may not all be exercised
> - Integration failures at layer boundaries are invisible to unit-level coverage
> - Risk-driven testing beats coverage-score maximizing every time

Hour 47 — Packaging and Delivery

Session 12, Hour 47

Hour 47 Learning Outcomes

By the end of this hour, learners will be able to:

- Explain the difference between "it runs on my machine" and "it ships"
- Produce `requirements.txt` and `requirements-dev.txt` from the current environment
- Document required environment variables without committing secret values
- Perform a fresh-run smoke test and fix one missing step if discovered

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour3_Advanced.md` → `## Learning Outcomes`

What Shipping Really Means

A project ships when **another person** can:

1. Clone or copy the source code
2. Follow the README without asking for hidden steps
3. Install dependencies from `requirements.txt`
4. Set documented environment variables
5. Run the app with one clear command
6. Run the test suite and see it pass

> "Shipping means someone else can run it without guessing."
>
> — `Advanced/lessons/lecture/Day12_Hour3_Advanced.md` → `### Talk points`

Minimum Deliverable Recipe

Every capstone submission needs these files:

File	Purpose
`README.md`	Setup, run, test, and artifact steps
`requirements.txt`	Runtime dependencies — pinned or recorded
`requirements-dev.txt`	Dev tools: `pytest`, `pytest-cov`
`.gitignore`	Excludes `.venv/`, `__pycache__/`, `*.db`
Entry point	One command that starts the chosen app surface

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour3_Advanced.md` → `### Talk points`

Demo: README Quickstart Sequence

```
# 1. Create and activate a virtual environment
python -m venv .venv
# Windows PowerShell: .venv\Scripts\Activate.ps1
# macOS or Linux: source .venv/bin/activate

# 2. Install runtime dependencies
python -m pip install -r requirements.txt

# 3. Install dev tools
python -m pip install -r requirements-dev.txt

# 4. Set required environment variables
# Windows PowerShell: $env:TRACKER_API_KEY="dev-key"
# macOS or Linux: export TRACKER_API_KEY=dev-key

# 5. Run the app
python -m api.app

# 6. Run the tests
pytest -q
```

Generating Dependency Files

```
# Capture all installed packages from the active venv
pip freeze > requirements.txt

# Review and trim: keep only what the app actually imports
# Move pytest and coverage tools into requirements-dev.txt
```

requirements-dev.txt content:

```
pytest
pytest-cov
```

Path safety rule

Use `pathlib.Path` for all file paths — never hard-code machine-specific paths.

```
from pathlib import Path

DB_PATH = Path(__file__).parent / "data" / "tracker.db"
```

Packaging Quality Gate

Before the lab, evaluate the capstone against this checklist:

Item	Ready evidence
README has setup + run + test steps	Reviewer can follow without asking
Dependencies are complete	No `ModuleNotFoundError` on fresh install
Environment variables are documented	No secret values committed to Git
Entry point is one clear command	`python -m api.app` or equivalent
`pytest -q` passes from a clean state	Tests do not depend on IDE memory

Lab: Make the Project Shareable (Hour 47)

Time — 25–35 minutes

Tasks

1. Update `README.md` with exact setup, run, and test commands
2. Run `pip freeze > requirements.txt` and review the output
3. Create `requirements-dev.txt` with `pytest` and `pytest-cov`
4. Update `.gitignore` to exclude `.venv/`, `__pycache__/`, and `*.db`
5. Perform a fresh-run smoke test from a clean terminal or new directory

Completion Criteria

- Another learner can follow the README without asking for hidden steps
- No machine-specific paths remain in the production code

Common Pitfalls (Hour 47)

⚠️ Missing a dependency in `requirements.txt` that happens to be installed globally

⚠️ Committing a `.env` file or credentials — document variable names, never commit values

⚠️ Entry-point path that only resolves inside the IDE's working directory

⚠️ Spending all packaging time on an optional executable before the basic release works

⚠️ README commands that only work with the original instructor environment assumptions

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour3_Advanced.md` → `### Common pitfalls to watch for`

Quick Check (Hour 47)

Question: What does a fresh-run test reveal that "it works on my machine" usually hides?

> **Discussion points:**
>
> - Packages installed globally but absent from `requirements.txt`
> - Hard-coded paths that only resolve in the original working directory
> - Environment variables that were set once and never documented
> - IDE-specific configuration that does not survive a new clone

Hour 48 — Final Demo, Review, and Retrospective

Session 12, Hour 48

Hour 48 Learning Outcomes

By the end of this hour, learners will be able to:

- Demonstrate an end-to-end working capstone system within a 3–5 minute timebox
- Answer architecture and tradeoff questions from a reviewer
- Practice certification-style code-reading on short Python prompts
- Create a concrete next-step practice plan after the course

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour4_Advanced.md` → `## Learning Outcomes`

Final Demo: Minimum Capstone Deliverables

Every demo should show evidence across all six layers:

Layer	Evidence
Model + Service	Domain object created, validated, returned
Persistence	Data reloaded across restart or a second query
Interface surface	GUI window **or** API JSON response shown
Analytics or Report	Generated file or printed summary shown
Tests	`pytest -q` output or a recent passing result
Packaging	README quickstart or `requirements.txt` shown

> "Coherence over perfection."
>
> — `Advanced/lessons/lecture/Day12_Hour4_Advanced.md` → `### Talk points`

Strong Demo Flow

Six steps — target 3–5 minutes total:

1. **Problem statement** — one sentence: what does the tracker solve?
2. **Create or retrieve data** — show the service layer in action
3. **Show persistence** — reload or re-query to prove SQLite saved the record
4. **Show an interface surface** — GUI window, API call, or CLI output
5. **Show one analytics or report artifact** — file path, chart, or summary
6. **Mention reliability evidence** — `pytest -q` result, coverage note, or packaging step

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour4_Advanced.md` → `### Demo steps`

Reviewer Prompts During Demos

Expect questions like these:

- "Why did you put that logic in the service rather than the route?"
- "What happens when a record ID does not exist?"
- "Which part was hardest to stabilize, and how did you solve it?"
- "What tradeoff did you make intentionally?"
- "What would you improve or add next?"

If something fails during the demo

State expected behavior → read the error message → name the diagnostic step.
Narrating a failure calmly is a professional skill, not a sign of weakness.

Source Alignment

Certification-Style Code Reading

```
def choose_label(status: str) -> str:
    if status in {"done", "closed"}:
        return "complete"
    if status:
        return "in progress"
    return "unknown"
```

Predict the output for each call before running:

```
print(choose_label("done"))     # ?
print(choose_label("open"))     # ?
print(choose_label(""))         # ?
print(choose_label("closed"))   # ?
```

Practice pattern: read carefully, state which branch is taken, then confirm.

Source Alignment

Certification-Style Code Reading — Answers

```
print(choose_label("done"))     # complete
print(choose_label("open"))     # in progress
print(choose_label(""))         # unknown   (empty string is falsy)
print(choose_label("closed"))   # complete
```

Key concepts exercised

- Set membership test with `in {"done", "closed"}`
- Truthiness of an empty string — `""` evaluates to `False`
- Early `return` from a function — later branches are never reached
- No mutation of the input parameter

Retrospective Prompts

Ask yourself or a study partner:

- What can you do now that you could not do before this course?
- What still feels shaky but understandable?
- What will you practice most in the first two weeks after today?

Post-course action choices

- Keep improving the capstone tracker — one new feature per week
- Read the pytest documentation on parameterization and marks
- Explore Flask Blueprints or SQLAlchemy as optional next layers
- Practice certification-style code reading with short daily snippets

Next-Step Planning Template

Write down four things before leaving today:

Prompt	Your answer
One skill to apply immediately at work or in study	
One Python concept to keep practicing	
One concrete capstone improvement to make this week	
The single command you will run when you return to the project	

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour4_Advanced.md` → `## Retrospective and Next-Step Planning`

Common Pitfalls (Hour 48)

⚠️ Trying to add or fix features during the final hour — stabilize and communicate what already works

⚠️ Demo overrun because setup was not rehearsed from the README beforehand

⚠️ Presenting code that was copied without being able to explain the design choices

⚠️ Ignoring a failure during the demo instead of narrating the debugging plan

⚠️ Finishing the course without a concrete next step written down

Source Alignment

- `Advanced/lessons/lecture/Day12_Hour4_Advanced.md` → `### Common pitfalls to watch for`

Quick Check (Hour 48)

Question: What makes a final demo convincing even when the project is not perfect?

> **Answer discussion:**
>
> - A clear problem statement and architecture explanation
> - Evidence of persistence — reload or re-query, not just a first run
> - Honest acknowledgment of one tradeoff or known gap
> - Calm, structured response if something fails during the demo
> - A specific next improvement named — not just "make it better"

Session 12 Summary

What We Built and Proved Today

Hour	Artifact	Evidence of success
45	`tests/test_service.py`	`pytest -q` — all green, sad paths covered
46	Hardened suite and coverage report	Risk-informed improvements, integration test passes
47	`README.md` and `requirements.txt`	Fresh-run smoke test succeeds on a clean install
48	Final capstone demo	Architecture explained, persistence shown, tests mentioned

Session theme

Course Wrap-Up — What You Can Now Do

After 48 hours across 12 sessions, you can:

- Design layered Python applications with clean, testable boundaries
- Persist and query structured data with SQLite through a repository
- Expose services through a Flask JSON API with typed error contracts
- Build analytics and reporting pipelines from persisted data
- Protect behavior with a focused pytest test suite using fixtures
- Package a runnable deliverable another developer can install and run
- Explain architecture choices and tradeoffs under review

> The goal was never to memorize syntax. It was to build systems that communicate clearly.

The Capstone Architecture in Full

```
advanced_tracker/
├── tracker/            <- domain model, service, repository
│   ├── model.py
│   ├── service.py
│   └── repository.py
├── api/                <- Flask routes and JSON serialization
│   └── app.py
├── reports/            <- analytics and report pipeline
│   └── generate_report.py
├── tests/              <- pytest test suite
│   └── test_service.py
├── requirements.txt
├── requirements-dev.txt
└── README.md
```

Every layer has a single job. Every boundary is testable independently.

Post-Course Practice Plan

Keep the capstone alive with one deliberate improvement per week:

- **Week 1** — Parameterize one group of pytest tests; add one edge case per layer
- **Week 2** — Add a second API endpoint or a second report format
- **Week 3** — Read the Flask or SQLAlchemy documentation for one new pattern
- **Week 4** — Walk a colleague through the README and explain one design tradeoff

Certification path

Code reading is a skill that improves with daily practice.
Predict the output of short Python snippets — before running — and explain the branch taken.

Final Exit Ticket

Before you leave today, write one sentence for each prompt:

1. The test that gives me the most confidence in my capstone is...
2. The thing a fresh-run test taught me about my project is...
3. The part of my capstone that best shows my growth across the course is...
4. The first thing I will do when I return to this project is...

> "Name what works, name what still needs attention, name your next command."
>
> — `Advanced/lessons/lecture/Day12_Hour4_Advanced.md` → `## Closing Debrief`